In Chapter 5 we carefully examined Python's array-based list class, and in Chapter 6 we demonstrated use of that class in implementing the classic stack, queue, and dequeue ADTs. Python's list class is highly optimized, and often a great choice for storage. With that said, there are some notable disadvantages:

1. The length of a dynamic array might be longer than the actual number of elements that it stores.
2. Amortized bounds for operations may be unacceptable in real-time systems.
3. Insertions and deletions at interior positions of an array are expensive.

In this chapter, we introduce a data structure known as a ***linked list***, which provides an alternative to an array-based sequence (such as a Python list). Both array-based sequences and linked lists keep elements in a certain order, but using a very different style. An array provides the more centralized representation, with one large chunk of memory capable of accommodating references to many elements. A linked list, in contrast, relies on a more distributed representation in which a lightweight object, known as a ***node***, is allocated for each element. Each node maintains a reference to its element and one or more references to neighboring nodes in order to collectively represent the linear order of the sequence.

We will demonstrate a trade-off of advantages and disadvantages when contrasting array-based sequences and linked lists. Elements of a linked list cannot be efficiently accessed by a numeric index k, and we cannot tell just by examining a node if it is the second, fifth, or twentieth node in the list. However, linked lists avoid the three disadvantages noted above for array-based sequences.

7.1 Singly Linked Lists

A ***singly linked list***, in its simplest form, is a collection of ***nodes*** that collectively form a linear sequence. Each node stores a reference to an object that is an element of the sequence, as well as a reference to the next node of the list (see Figures 7.1 and 7.2).

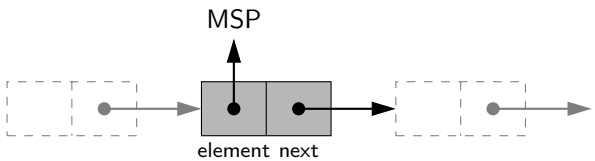


Figure 7.1: Example of a node instance that forms part of a singly linked list. The node's element member references an arbitrary object that is an element of the sequence (the airport code MSP, in this example), while the next member references the subsequent node of the linked list (or None if there is no further node).

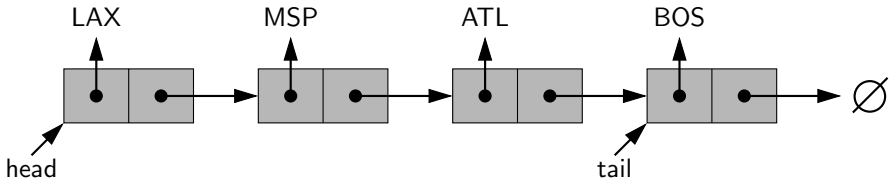


Figure 7.2: Example of a singly linked list whose elements are strings indicating airport codes. The list instance maintains a member named head that identifies the first node of the list, and in some applications another member named tail that identifies the last node of the list. The **None** object is denoted as Ø.

The first and last node of a linked list are known as the ***head*** and ***tail*** of the list, respectively. By starting at the head, and moving from one node to another by following each node's next reference, we can reach the tail of the list. We can identify the tail as the node having None as its next reference. This process is commonly known as ***traversing*** the linked list. Because the next reference of a node can be viewed as a ***link*** or ***pointer*** to another node, the process of traversing a list is also known as ***link hopping*** or ***pointer hopping***.

A linked list's representation in memory relies on the collaboration of many objects. Each node is represented as a unique object, with that instance storing a reference to its element and a reference to the next node (or None). Another object represents the linked list as a whole. Minimally, the linked list instance must keep a reference to the head of the list. Without an explicit reference to the head, there would be no way to locate that node (or indirectly, any others). There is not an absolute need to store a direct reference to the tail of the list, as it could otherwise be located by starting at the head and traversing the rest of the list. However, storing an explicit reference to the tail node is a common convenience to avoid such a traversal. In similar regard, it is common for the linked list instance to keep a count of the total number of nodes that comprise the list (commonly described as the ***size*** of the list), to avoid the need to traverse the list to count the nodes.

For the remainder of this chapter, we continue to illustrate nodes as objects, and each node's "next" reference as a pointer. However, for the sake of simplicity, we illustrate a node's element embedded directly within the node structure, even though the element is, in fact, an independent object. For example, Figure 7.3 is a more compact illustration of the linked list from Figure 7.2.

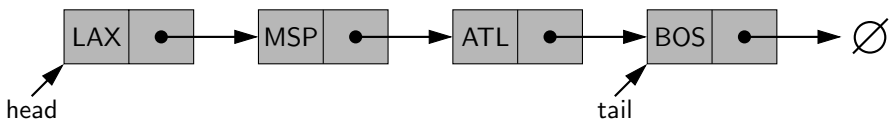


Figure 7.3: A compact illustration of a singly linked list, with elements embedded in the nodes (rather than more accurately drawn as references to external objects).

Inserting an Element at the Head of a Singly Linked List

An important property of a linked list is that it does not have a predetermined fixed size; it uses space proportionally to its current number of elements. When using a singly linked list, we can easily insert an element at the head of the list, as shown in Figure 7.4, and described with pseudo-code in Code Fragment 7.1. The main idea is that we create a new node, set its element to the new element, set its next link to refer to the current head, and then set the list's head to point to the new node.

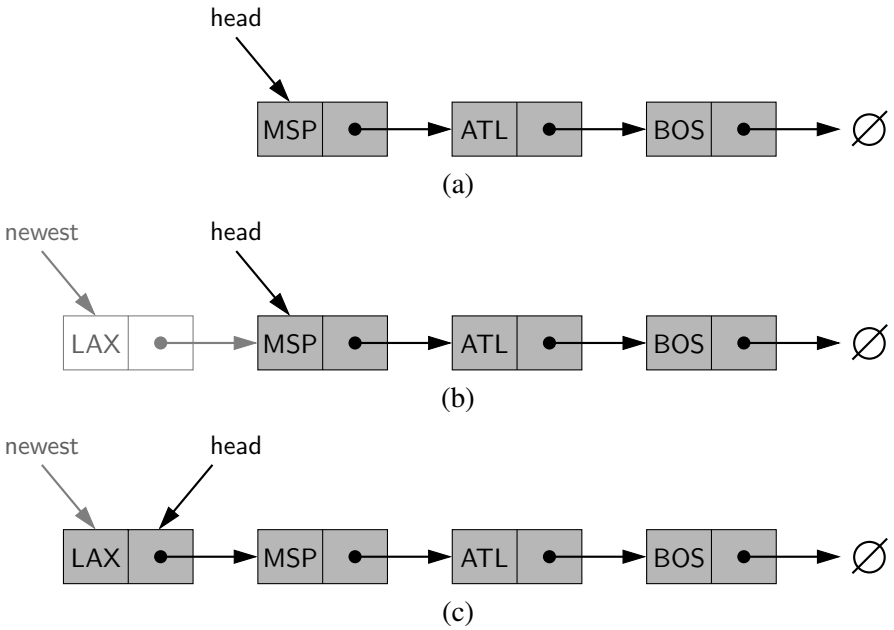


Figure 7.4: Insertion of an element at the head of a singly linked list: (a) before the insertion; (b) after creation of a new node; (c) after reassignment of the head reference.

Algorithm add_first(L, e):

```
newest = Node(e)      {create new node instance storing reference to element e}
newest.next = L.head  {set new node's next to reference the old head node}
L.head = newest       {set variable head to reference the new node}
L.size = L.size + 1   {increment the node count}
```

Code Fragment 7.1: Inserting a new element at the beginning of a singly linked list L. Note that we set the next pointer of the new node ***before*** we reassign variable L.head to it. If the list were initially empty (i.e., L.head is None), then a natural consequence is that the new node has its next reference set to None.

Inserting an Element at the Tail of a Singly Linked List

We can also easily insert an element at the tail of the list, provided we keep a reference to the tail node, as shown in Figure 7.5. In this case, we create a new node, assign its next reference to None, set the next reference of the tail to point to this new node, and then update the tail reference itself to this new node. We give the details in Code Fragment 7.2.

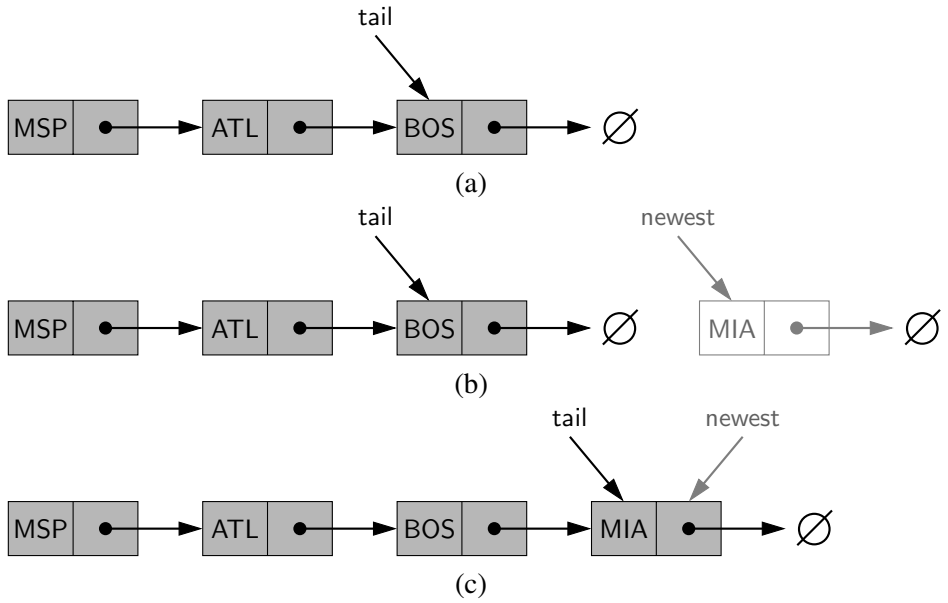


Figure 7.5: Insertion at the tail of a singly linked list: (a) before the insertion; (b) after creation of a new node; (c) after reassignment of the tail reference. Note that we must set the next link of the tail in (b) before we assign the tail variable to point to the new node in (c).

Algorithm add_last(L, e):

```
newest = Node(e)      {create new node instance storing reference to element e}
newest.next = None    {set new node's next to reference the None object}
L.tail.next = newest  {make old tail node point to new node}
L.tail = newest       {set variable tail to reference the new node}
L.size = L.size + 1   {increment the node count}
```

Code Fragment 7.2: Inserting a new node at the end of a singly linked list. Note that we set the next pointer for the old tail node ***before*** we make variable tail point to the new node. This code would need to be adjusted for inserting onto an empty list, since there would not be an existing tail node.

Removing an Element from a Singly Linked List

Removing an element from the ***head*** of a singly linked list is essentially the reverse operation of inserting a new element at the head. This operation is illustrated in Figure 7.6 and given in detail in Code Fragment 7.3.

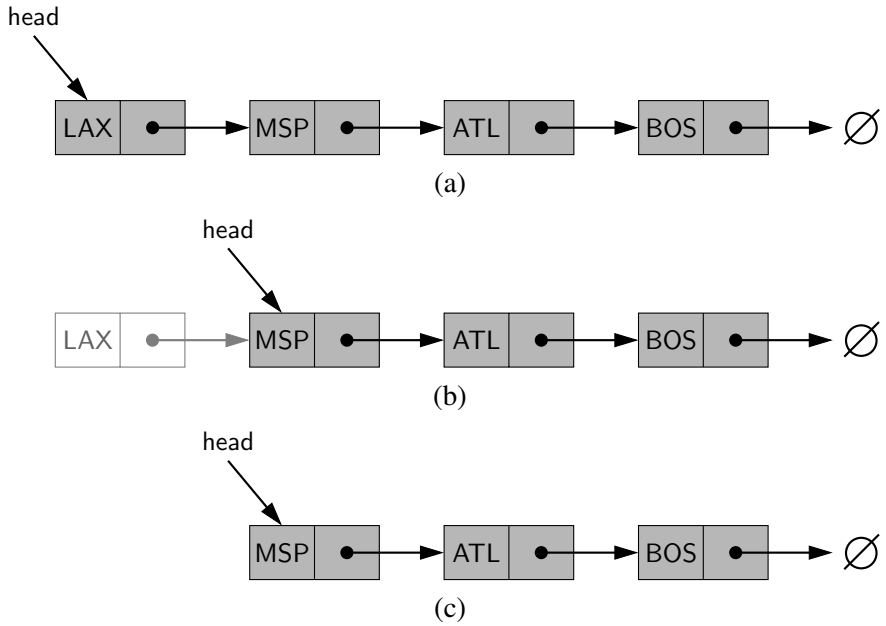


Figure 7.6: Removal of an element at the head of a singly linked list: (a) before the removal; (b) after "linking out" the old head; (c) final configuration.

Algorithm remove_first(L):
  if L.head is None **then**
    Indicate an error: the list is empty.
  L.head = L.head.next {make head point to next node (or None)}
  L.size = L.size − 1 {decrement the node count}

Code Fragment 7.3: Removing the node at the beginning of a singly linked list.

Unfortunately, we cannot easily delete the last node of a singly linked list. Even if we maintain a tail reference directly to the last node of the list, we must be able to access the node ***before*** the last node in order to remove the last node. But we cannot reach the node before the tail by following next links from the tail. The only way to access this node is to start from the head of the list and search all the way through the list. But such a sequence of link-hopping operations could take a long time. If we want to support such an operation efficiently, we will need to make our list ***doubly linked*** (as we do in Section 7.3).

7.1.1 Implementing a Stack with a Singly Linked List

In this section, we demonstrate use of a singly linked list by providing a complete Python implementation of the stack ADT (see Section 6.1). In designing such an implementation, we need to decide whether to model the top of the stack at the head or at the tail of the list. There is clearly a best choice here; we can efficiently insert and delete elements in constant time only at the head. Since all stack operations affect the top, we orient the top of the stack at the head of our list.

To represent individual nodes of the list, we develop a lightweight _Node class. This class will never be directly exposed to the user of our stack class, so we will formally define it as a nonpublic, nested class of our eventual LinkedStack class (see Section 2.5.1 for discussion of nested classes). The definition of the _Node class is shown in Code Fragment 7.4.

```
class _Node:
  """Lightweight, nonpublic class for storing a singly linked node."""
  __slots__ = '_element', '_next'          # streamline memory usage

  def __init__(self, element, next):       # initialize node's fields
    self._element = element                # reference to user's element
    self._next = next                      # reference to next node
```

Code Fragment 7.4: A lightweight _Node class for a singly linked list.

A node has only two instance variables: _element and _next. We intentionally define __slots__ to streamline the memory usage (see page 99 of Section 2.5.1 for discussion), because there may potentially be many node instances in a single list. The constructor of the _Node class is designed for our convenience, allowing us to specify initial values for both fields of a newly created node.

A complete implementation of our LinkedStack class is given in Code Fragments 7.5 and 7.6. Each stack instance maintains two variables. The _head member is a reference to the node at the head of the list (or None, if the stack is empty). We keep track of the current number of elements with the _size instance variable, for otherwise we would be forced to traverse the entire list to count the number of elements when reporting the size of the stack.

The implementation of push essentially mirrors the pseudo-code for insertion at the head of a singly linked list as outlined in Code Fragment 7.1. When we push a new element e onto the stack, we accomplish the necessary changes to the linked structure by invoking the constructor of the _Node class as follows:

```
self._head = self._Node(e, self._head)      # create and link a new node
```

Note that the _next field of the new node is set to the ***existing*** top node, and then self._head is reassigned to the new node.

```
1  class LinkedStack:
2    """LIFO Stack implementation using a singly linked list for storage."""
3
4    #---------------------------- nested _Node class ----------------------------
5    class _Node:
6      """Lightweight, nonpublic class for storing a singly linked node."""
7      __slots__ = '_element', '_next'         # streamline memory usage
8
9      def __init__(self, element, next):      # initialize node's fields
10       self._element = element               # reference to user's element
11       self._next = next                     # reference to next node
12
13   #------------------------------- stack methods -------------------------------
14   def __init__(self):
15     """Create an empty stack."""
16     self._head = None                       # reference to the head node
17     self._size = 0                          # number of stack elements
18
19   def __len__(self):
20     """Return the number of elements in the stack."""
21     return self._size
22
23   def is_empty(self):
24     """Return True if the stack is empty."""
25     return self._size == 0
26
27   def push(self, e):
28     """Add element e to the top of the stack."""
29     self._head = self._Node(e, self._head)    # create and link a new node
30     self._size += 1
31
32   def top(self):
33     """Return (but do not remove) the element at the top of the stack.
34
35     Raise Empty exception if the stack is empty.
36     """
37     if self.is_empty():
38       raise Empty('Stack is empty')
39     return self._head._element              # top of stack is at head of list
```

Code Fragment 7.5: Implementation of a stack ADT using a singly linked list (continued in Code Fragment 7.6).

```
40    def pop(self):
41      """Remove and return the element from the top of the stack (i.e., LIFO).
42
43      Raise Empty exception if the stack is empty.
44      """
45      if self.is_empty( ):
46        raise Empty('Stack is empty')
47      answer = self._head._element
48      self._head = self._head._next              # bypass the former top node
49      self._size -= 1
50      return answer
```

Code Fragment 7.6: Implementation of a stack ADT using a singly linked list (continued from Code Fragment 7.5).

When implementing the top method, the goal is to return the ***element*** that is at the top of the stack. When the stack is empty, we raise an Empty exception, as originally defined in Code Fragment 6.1 of Chapter 6. When the stack is nonempty, self._head is a reference to the first ***node*** of the linked list. The top element can be identified as self._head._element.

Our implementation of pop essentially mirrors the pseudo-code given in Code Fragment 7.3, except that we maintain a local reference to the element that is stored at the node that is being removed, and we return that element to the caller of pop.

The analysis of our LinkedStack operations is given in Table 7.1. We see that all of the methods complete in ***worst-case*** constant time. This is in contrast to the amortized bounds for the ArrayStack that were given in Table 6.2.

Operation	Running Time
S.push(e)	$O(1)$
S.pop( )	$O(1)$
S.top( )	$O(1)$
len(S)	$O(1)$
S.is_empty( )	$O(1)$

Table 7.1: Performance of our LinkedStack implementation. All bounds are worst-case and our space usage is $O(n)$, where n is the current number of elements in the stack.

7.1.2 Implementing a Queue with a Singly Linked List

As we did for the stack ADT, we can use a singly linked list to implement the queue ADT while supporting worst-case $O(1)$-time for all operations. Because we need to perform operations on both ends of the queue, we will explicitly maintain both a _head reference and a _tail reference as instance variables for each queue. The natural orientation for a queue is to align the front of the queue with the head of the list, and the back of the queue with the tail of the list, because we must be able to enqueue elements at the back, and dequeue them from the front. (Recall from the introduction of Section 7.1 that we are unable to efficiently remove elements from the tail of a singly linked list.) Our implementation of a LinkedQueue class is given in Code Fragments 7.7 and 7.8.

```
1  class LinkedQueue:
2    """FIFO queue implementation using a singly linked list for storage."""
3
4    class _Node:
5      """Lightweight, nonpublic class for storing a singly linked node."""
6      (omitted here; identical to that of LinkedStack._Node)
7
8    def __init__(self):
9      """Create an empty queue."""
10     self._head = None
11     self._tail = None
12     self._size = 0                          # number of queue elements
13
14   def __len__(self):
15     """Return the number of elements in the queue."""
16     return self._size
17
18   def is_empty(self):
19     """Return True if the queue is empty."""
20     return self._size == 0
21
22   def first(self):
23     """Return (but do not remove) the element at the front of the queue."""
24     if self.is_empty( ):
25       raise Empty('Queue is empty')
26     return self._head._element              # front aligned with head of list
```

Code Fragment 7.7: Implementation of a queue ADT using a singly linked list (continued in Code Fragment 7.8).

```
27    def dequeue(self):
28      """Remove and return the first element of the queue (i.e., FIFO).
29
30      Raise Empty exception if the queue is empty.
31      """
32      if self.is_empty( ):
33        raise Empty('Queue is empty')
34      answer = self._head._element
35      self._head = self._head._next
36      self._size -= 1
37      if self.is_empty( ):                    # special case as queue is empty
38        self._tail = None                     # removed head had been the tail
39      return answer
40
41    def enqueue(self, e):
42      """Add an element to the back of queue."""
43      newest = self._Node(e, None)            # node will be new tail node
44      if self.is_empty( ):
45        self._head = newest                   # special case: previously empty
46      else:
47        self._tail._next = newest
48      self._tail = newest                     # update reference to tail node
49      self._size += 1
```

Code Fragment 7.8: Implementation of a queue ADT using a singly linked list (continued from Code Fragment 7.7).

Many aspects of our implementation are similar to that of the LinkedStack class, such as the definition of the nested _Node class. Our implementation of dequeue for LinkedQueue is similar to that of pop for LinkedStack, as both remove the head of the linked list. However, there is a subtle difference because our queue must accurately maintain the _tail reference (no such variable was maintained for our stack). In general, an operation at the head has no effect on the tail, but when dequeue is invoked on a queue with one element, we are simultaneously removing the tail of the list. We therefore set self._tail to None for consistency.

There is a similar complication in our implementation of enqueue. The newest node always becomes the new tail. Yet a distinction is made depending on whether that new node is the only node in the list. In that case, it also becomes the new head; otherwise the new node must be linked immediately after the existing tail node.

In terms of performance, the LinkedQueue is similar to the LinkedStack in that all operations run in worst-case constant time, and the space usage is linear in the current number of elements.